

1. Two Sum

Solved ✓

Easy

Topics

Companies

Hint

Given an array of integers `nums` and an integer `target`, return indices of the two numbers such that they add up to `target`.

You may assume that each input would have **exactly one solution**, and you may not use the same element twice.

You can return the answer in any order.

Example 1:

Input: `nums = [2,7,11,15]`, `target = 9`

Output: `[0,1]`

Explanation: Because `nums[0] + nums[1] == 9`, we return `[0, 1]`.

0	1	2	3
2	7	11	15

target = 9

0 | 1

3 | 4 | 5

6
target

Simplest Testcase

0	1	2	3	4
7	10	20	15	30

target = 45

$i \rightarrow 0$ to $\text{len}(\text{nums}) - 1$
 $j \rightarrow i + 1$ to $\text{len}(\text{nums})$

We need 2 elements that add to target

$$15 + 30 = 45$$

o/p

3 | 4

i	j	nums[i]	nums[j]
0	1	7	10
0	2	7	20
0	3	7	15
0	4	7	30
1	2	10	20
1	3	10	15
1	4	10	30
2	3	20	15
2	4	20	30
3	4	15	30

```
class Solution:
```

```
def twoSum(self, nums: List[int], target: int) -> List[int]:
    for i in range(0, len(nums) - 1):
        for j in range(i + 1, len(nums)):
            if nums[i] + nums[j] == target:
                return [j, i]
```

$\downarrow$
 p.no: (i)
 $\downarrow$
 $T.C \Rightarrow O(n^2)$
 $\downarrow$
 $O.C \Rightarrow O(1)$
 $(n-1) + (n-2) + (n-3) + (n-4) + \dots + 2 + 1$
 $n \times (n-1) \Rightarrow n^2 - n$

$O(n^2)$ $\rightarrow$ Time complexity

0	1	2	3	4
10	20	40	25	30

target: 55

10	:	0
20	:	1
40	:	2
25	:	3

3	4
---	---

$\rightarrow$ Dictionary

$\rightarrow$ store key value pairs

```

# Creating a dictionary with key and value pairs
dict = {
    50 : 8,
    100 : 0,
}

if 10 in dict :
    print("10 is present")
else :
    print("10 is not present")

for key in dict :
    print(key, " : ", dict[key])

print("Before adding 50 : " , dict[50])
dict[50] = 10 # Adding an key value pair to dictionary, if key is present
               -> update the value, else -> create a new key value pair
print("After adding 50 : " , dict[50])
    
```

0	1	2	3	4
100	20	40	70	90

$4 \times 3 = 12$

target: 160

100 : 0
20 : 1
40 : 2
70 : 3

$\left[\frac{1}{0} \right]$

$\left[\frac{4}{3} \right] \rightarrow 0/1$

class Solution:

```
def twoSum(self, nums: List[int], target: int) -> List[int]:
    seen = {}
    for i in range(len(nums)):
        elem = target - nums[i]
        if elem in seen:
            j = seen[elem]
            return [i, j]
        seen[nums[i]] = i
```

↓
p.No : 1

↓
T.C $\rightarrow O(n)$

↓
s.c $\rightarrow O(n)$

217. Contains Duplicate

Solved

Easy Topics Companies

Given an integer array nums, return true if any value appears at least twice in the array, and return false if every element is distinct.

Example 1:

Input: nums = [1, 2, 3, 1]

Output: true

5 | 2 | 15 | 7

↓
False

10 | 40 | 30 | 20
0 1 2 3

i j $nums[i]$ $nums[j]$
0 1 10 10

False

0	1	2	3
10	20	40	20

True ✓

i	j	nums[i]	nums[j]
0	2	10	X 30
0	3	10	X 20
1	2	40	X 30
1	3	40	X 20
2	3	30	X 20

i	j	nums[i]	nums[j]
0	1	10	X 20
0	2	10	X 40
0	3	10	= 10

```
class Solution:
    def containsDuplicate(self, nums: List[int]) -> bool:
        for i in range(0, len(nums) - 1):
            for j in range(i + 1, len(nums)):
                if nums[i] == nums[j]:
                    return True
        return False
```

→ T.C → $O(n^2)$

Call Stack → Not empty → 2 seconds → TLE

1 sec → 10^4 operations, 2 sec → 10^5 operations

Two Sum

Contains Duplicate

$2 \leq n \leq 10^4$

$1 \leq n \leq 10^5$

T.C → $O(n^2)$ → $(10^4)^2$

$n^2 \rightarrow (10^5)^2 \rightarrow 10^{10}$

2 sec → 10^5

TLE → $2.5 > 2 \text{ sec}$


```
# Sets
nums = set() # Creates an empty set
nums = { 20, 40, 20}
if 200 in nums :
    print("Yes")
else :
    print("No")

nums.add(70) # Adds 70 to the set
nums.discard(10) # Removes the element, if present, otherwise
                 # doesn't return any Error
nums.remove(10) # Removes element from set, if element not
                # present -> KeyError
print(nums, type(nums))
```

```
class Solution:
    def containsDuplicate(self, nums: List[int]) -> bool:
        seen = set()
        for i in nums :
            if i in seen :
                return True
            seen.add(i)
        return False
```

P.No: 217

T.C $\rightarrow O(n)$

S.C $\rightarrow O(n)$

```
class Solution:
    def containsDuplicate(self, nums: List[int]) -> bool:
        return len(set(nums)) != len(nums)
```

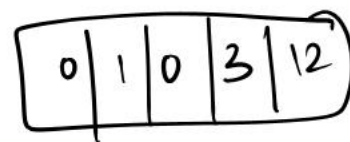
283. Move Zeroes

Solved 

Easy Topics Companies Hint

Given an integer array nums, move all 0's to the end of it while maintaining the relative order of the non-zero elements.

Note that you must do this in-place without making a copy of the array.



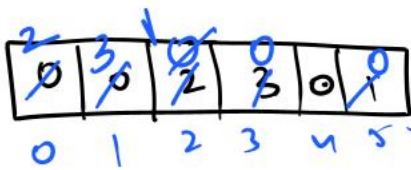
Example 1:

Example 1.

Input: nums = [0,1,0,3,12]

Output: [1,3,12,0,0]

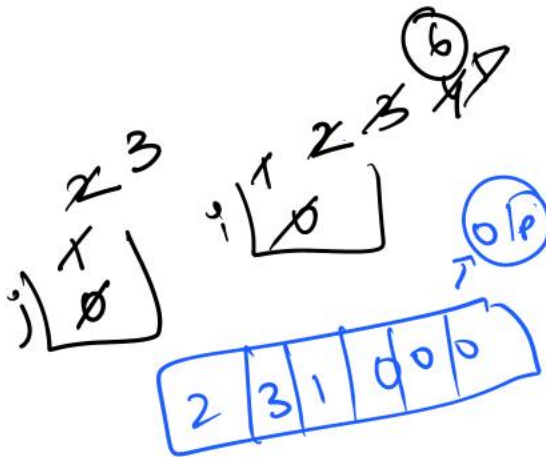
(O/r)



1st Non zero $\rightarrow 0$

2nd Non zero $\rightarrow 1$

3rd Non zero $\rightarrow 2$



$0 \leq \text{No. of Non zero elements} \leq n$

class Solution:

def moveZeroes(self, nums: List[int]) -> None:

"""

Do not return anything, modify nums in-place instead.

"""

j = 0

for i in range(len(nums)) :

if nums[i] != 0 :

nums[i], nums[j] = nums[j], nums[i]

j += 1

↓
2 3 2

T.C : $O(n)$

S.C $\rightarrow O(1)$

Q. No. 121

121. Best Time to Buy and Sell Stock

Solved ✓

Easy Topics Companies

You are given an array `prices` where `prices[i]` is the price of a given stock on the i^{th} day.

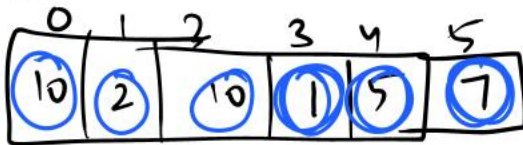
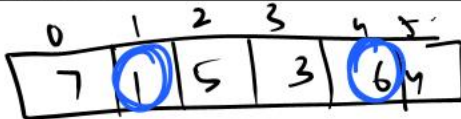
You want to maximize your profit by choosing a **single day** to buy one stock and choosing a **different day in the future** to sell that stock.

Return the maximum profit you can achieve from this transaction. If you cannot achieve any profit, return 0.

Example 1:

Input: `prices = [7,1,5,3,6,4]`

Output: 5



Either Buy or Selling

max $\begin{cases} 8 \\ 0 \end{cases}$ $\rightarrow$ 0/p

$6 - 1 = 5 \rightarrow$ o/p

i	buy	sell	profit
0	10	-	-
1	2	-	-
2	2	10	8
3	1	-	-
4	1	5	4
5	1	7	6

class Solution:

```
def maxProfit(self, prices: List[int]) -> int:
    output = 0
    buyingPrice = prices[0]
    for i in range(1, len(prices)):
        # Buy -> currentPrice < buyingPrice
        if prices[i] < buyingPrice:
            buyingPrice = prices[i]
```

$\rightarrow$ Q. No. 121

$\rightarrow$ T.C $\rightarrow O(n)$

$\rightarrow$ S.C $\rightarrow O(1)$

```

else :
    profit = prices[i] - buyingPrice
    output = max(profit, output)
return output

```

→ 3 2 1 0

4

0	0	0		0	0	0	^	0	0	0
0	1	0		0	1	1		0	1	1
1	0	0		1	0	1		1	0	1
1	1	1		1	1	1		1	1	0

10 & 8

	2^3	2^2	2^1	2^0
10 →	1	0	1	0
8 →	1	0	0	0
	1	0	0	0
	→ 8			

136. Single Number

Solved ✓

Easy

Topics

Companies

Hint

Given a **non-empty** array of integers **nums**, every element appears **twice** except for one. Find that **single one**.

You must implement a solution with a **linear runtime complexity** and use only **constant extra space**.

↓
s.c $O(1)$

↓
T.C $O(n)$

Example 1:

Input: nums = [2,2,1]

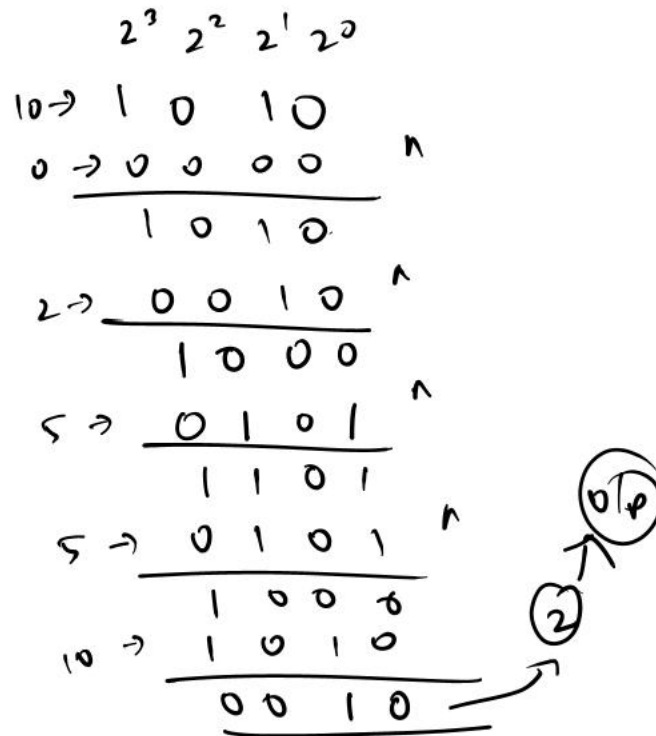
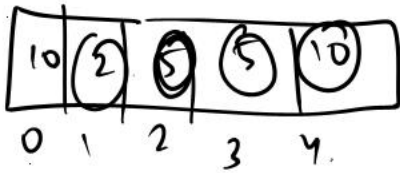
Output: 1

2 2 1

o/p : 1

5 2 1 1 5

o/p : 2



class Solution:

```
def singleNumber(self, nums: List[int]) -> int:
    ans = 0
    for i in nums :
        ans ^= i
    return ans
```

P.Wo: (136) T.C $\rightarrow O(n)$ S.C $\rightarrow O(1)$

905. Sort Array By Parity

Solved ✓

Easy Topics Companies

Given an integer array nums, move all the even integers at the beginning of the array followed by all the odd integers.

Return any array that satisfies this condition.



Even $\rightarrow$ 0 index

Odd $\rightarrow$ 1st index

Example 1:

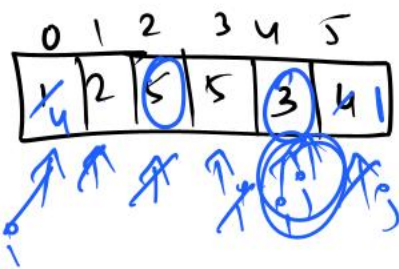
Input: `nums = [3,1,2,4]`

Output: `[2,4,3,1]`

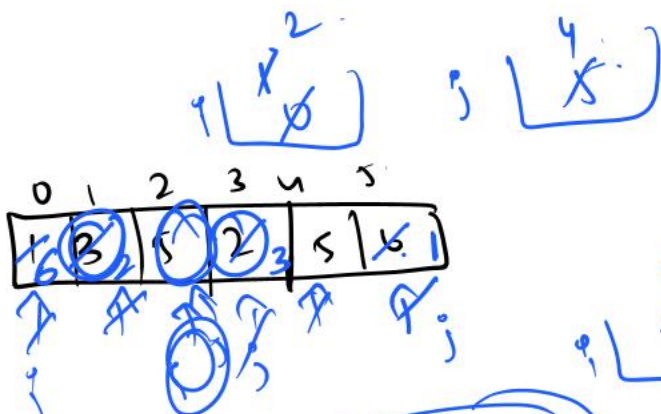
Explanation: The outputs `[4,2,3,1]`, `[2,4,1,3]`, and `[4,2,1,3]` would also be accepted.

class Solution:

```
def sortArrayByParity(self, nums: List[int]) -> List[int]:  
    j = 0  
    for i in range(len(nums)) :  
        if nums[i] % 2 == 0 :  
            nums[i], nums[j] = nums[j], nums[i]  
            j += 1  
    return nums
```



Move all even no's to front



$i \geq j \rightarrow \text{stop}$

$i < j \rightarrow \text{check for +ve and -ve no's}$

```

class Solution:
    def sortByParity(self, nums: List[int]) -> List[int]:
        i = 0
        j = len(nums) - 1
        while i < j:
            if nums[i] % 2 == 0:
                i += 1
            elif nums[j] % 2 == 1:
                j -= 1
            else:
                nums[i], nums[j] = nums[j], nums[i]
                i += 1
                j -= 1
        return nums

```

→ P.NO. 905

→ T.C $O(n)$

→ S.C $O(1)$

$k + n - k \Rightarrow n$ → time

941. Valid Mountain Array

Solved ✓

Easy

Topics

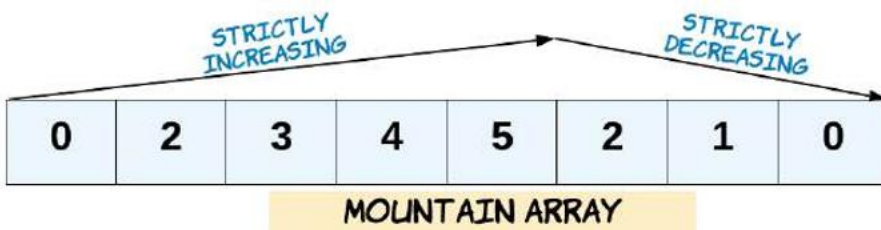
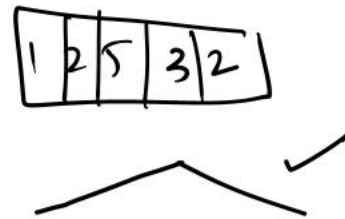
Companies

Hint

Given an array of integers arr, return true if and only if it is a valid mountain array.

Recall that arr is a mountain array if and only if:

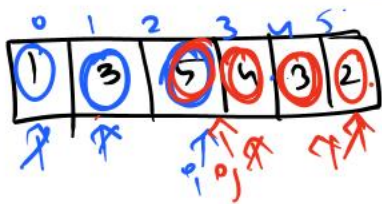
- arr.length >= 3
- There exists some i with $0 < i < \text{arr.length} - 1$ such that:
 - arr[0] < arr[1] < ... < arr[i - 1] < arr[i]
 - arr[i] > arr[i + 1] > ... > arr[arr.length - 1]





→ false

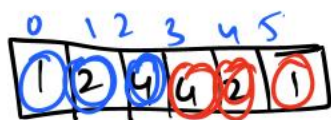
NOT A MOUNTAIN ARRAY



→ True. $i = j \rightarrow \text{True}$

strictly increasing part $\rightarrow i$ $\rightarrow \text{arr}[i] < \text{arr}[i+1]$

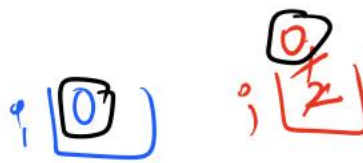
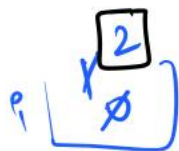
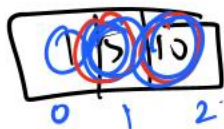
strictly decreasing part $\rightarrow j$ $\rightarrow \text{arr}[j] < \text{arr}[j-1]$



$i \neq j \rightarrow \text{False}$

strictly increasing $\rightarrow i$ $\rightarrow \text{arr}[i] < \text{arr}[i+1]$

strictly decreasing $\rightarrow j$ $\rightarrow \text{arr}[j] < \text{arr}[j-1]$



$i = j \rightarrow \text{False}$

```
class Solution:
    def validMountainArray(self, arr: List[int]) -> bool:
        # Strictly increasing -> 0 to len(arr) - 1
```

→ PNB: 941


```

i = 0
while i < len(arr) - 1 and arr[i] < arr[i + 1] :
    i += 1
# Strictly decreasing -> len(arr) - 1 to 1
j = len(arr) - 1
while j >= 1 and arr[j] < arr[j - 1] :
    j -= 1
if i != j or i == 0 or j == len(arr) - 1:
    return False
else :
    return True

```

→ T.C → $O(n)$

→ S.C → $O(1)$

→ Increasing or Decreasing slope

Not a Mountain

```
str1 = "Hello all"
```

```
print(str1.strip()) # Returns a string after removing all the trailing spaces in right and left.
```

```
print(str1.rstrip()) # Returns a string after removing all the trailing spaces in right.
```

```
print(str1.lstrip()) # Returns a string after removing all the trailing spaces in left.
```

```
str1 = str1.lower() # Converts into a string of lowercase and returns a String.
```

```
print(str1.isalnum()) # Checks if string is alphanumeric or not, if yes -> True, else -> False
```

```
print(str1.index("l")) # Returns the first occurrence of substring in the string, if not present -> throws ValueError
```

```
print(str1.count("l")) # Returns the count of sub strings present in string -> int
```

```
print(str1[:5]) # Prints substring from start to 5, 5 is exclusive in nature
```

```
print(str1[2:]) # Prints substring from 2 to end of string, 2 is inclusive in nature
```

```
print(str1[0:3]) # Prints substring from 0 to 3, 3 is exclusive, 0 is inclusive in nature
```

```
print(str1[0]) # Print the character, at index 0
```



```
    c1 = s[j].lower()
    if c != c1 :
        return False
    i += 1
    j -= 1
return True
```